Tecnologia-y-Arquitectura-Robotica / practica_1_2526

Code Issues Pull requests Agents Actions Projects Security Insights

Watch 0

1 star 0 forks 0 watching Branches Activity Custom properties Tags

Private repository

main 1 Branch 0 Tags Go to file Go to file Code

TamaiRamirezUA Practica 1 7bba095 · 2 weeks ago

servicio_suma	Practica 1	2 weeks ago
Dockerfile	Updating...	3 weeks ago
Instalacion.md	Practica 1	2 weeks ago
README.md	Practica 1	2 weeks ago
connect_ros.sh	Updating...	3 weeks ago
run.sh	Updating...	3 weeks ago

Práctica 1: Aprendiendo las bases de ROS 2 (Humble)

Índice

1. Entrega
2. Instalación
3. Parte 1: Primeros Pasos
   i. Creación del espacio de Trabajo
   ii. Ejemplo de un entorno
   iii. Servicios
   iv. Ejercicios
4. Parte 2: Nivel principiante
   i. Crear un Paquete
   ii. Crear un mensaje
   iii. Ejercicios
5. Parte 3: Pasando a nivel Intermedio
   i. Crear un archivo launch
   ii. Acciones
   iii. Ejercicios

Entrega

La entrega de esta práctica se realizará a través de la herramienta de 'Evaluación' de UaCloud. La misma debe ser una memoria en formato `.pdf` en la cual se encuentren las respuestas a las preguntas teóricas y a los ejercicios propuestos de las diferentes partes que conforman este `README.md`. Esta, deberá tener el nombre de *Apellidos_Nombre.pdf*. En esta práctica se deben entregar también los códigos que se hayan generado para resolver los ejercicios. Podéis o bien compartir un enlace al repositorio de `GitHub` que estéis usando, o añadirnos como colaboradores (Nuestros usuarios de github son TamaiRamirezUA y bigpacopujol, aunque nos añadáis como colaboradores, añadid el enlace al repositorio en la memoria) o compartid enlace de `Drive`, como prefiráis. Por otro lado, es recomendable que grabéis la resolución de los ejercicios y compartáis los videos en la memoria a través de un enlace también, así se puede observar la correcta ejecución de los ejercicios, no obstante, para esta práctica en concreto con presentar capturas de pantalla de la ejecución es suficiente. La entrega de esta práctica es **individual**.

Instalación

Para la realización de esta práctica y las sucesivas, se han generado varios `Dockerfile` que serán los encargados de generar las imágenes de ROS2 para poder trabajar con ellas. Esto nos va a permitir generar contenedores de `Docker` con dichas imágenes, de forma que, podamos tener todas las dependencias y librerías contenidas dentro de nuestro sistema operativo (Ubuntu 24.04 LTS u otro Linux-based), evitando así incompatibilidades con otras aplicaciones.

Con el fin de poder trabajar con los contenedores de `Docker` y ROS2 es necesario antes de comenzar, realizar los pasos que se encuentran en el documento Instalacion.md dentro de este repositorio. Este documento contiene la guía de instalación con las herramientas necesarias para poder ejecutar las prácticas de la asignatura.

A continuación, seguiremos con la instalación de ROS2 (Humble) para lo cual se requiere abrir una terminal dentro de la raíz de este repositorio (copialo con un `git clone`) y ejecutar los siguientes comandos que generaran la imágen de `Docker`:

1. Crear y construir la imágen de ROS2 (Humble) a través del `Dockerfile`:

```
docker build --build-arg USERNAME=$(whoami)-docker --build-arg USER_UID=$(id -u) -t ros_humble:latest .
```

Este comando comenzará la instalación de ROS2 en su Distro Humble.

2. Una vez instalado ROS2 ya podemos ejecutar el contenedor y conectarnos a él de la siguiente manera **sin usar Nvidia-Docker**:

```
sudo chmod u+x run.sh # Solo la primera vez para dar los permisos necesarios
./run.sh
```

Ya estaremos dentro del contenedor y podremos empezar a trabajar con ROS2.

3. Para conectarnos al contenedor desde nuevas terminales una vez lanzado el contenedor, ejecuta:

```
sudo chmod u+x connect_ros.sh # Solo la primera vez para dar los permisos necesarios
./connect_ros.sh
```

Parte 1: Primeros Pasos

Creación del espacio de Trabajo

Cuando se trabaja con el código fuente de ROS2, a menudo es útil hacerlo en un `workspace` (Espacio de trabajo). Para crear un espacio de trabajo ROS lo único que tenemos que hacer es navegar hasta el directorio en el que queramos crear el espacio de trabajo y ejecutar los comandos que aparecen a continuación. *Se recomienda crearlo en el directorio raíz de nuestro espacio de trabajo en el contenedor de `Docker` y con el nombre `ros2_ws`, de la forma siguiente*:

```
mkdir -p ros2_ws/src
cd ros2_ws/
```

Previamente a la construcción del espacio de trabajo, es necesario resolver las dependencias de los paquetes. Es posible que se disponga de todas las dependencias tras la instalación, sin embargo, es recomendable verificar al menos la primera vez que se genera el espacio de trabajo. Para llevar a cabo el proceso, es necesario acceder al espacio de trabajo (`ros2_ws`) y ejecutar los comandos siguientes:

```
sudo rosdep init
rosdep update
rosdep install -i --from-path src --rosdistro humble -y
```

Desde la raíz del espacio de trabajo (`ros2_ws`), ahora puedes construir sus paquetes utilizando el comando:

```
colcon build
```

A continuación, es necesario superponer este espacio de trabajo al entorno raíz de ROS 2, para ello ejecuta desde la raíz del espacio de trabajo (`ros2_ws`):

```
source /opt/ros/humble/setup.bash
source install/local_setup.bash
```

IMPORTANTE: Cada vez que entremos en el contenedor de ROS 2 ya sea ejecutando `./run.sh` o si accedemos al contenedor, una vez lanzado por el script anterior, en nuevas terminales para ejecutar diferentes herramientas de ROS ejecutando `./connect_ros.sh`, necesitamos que nuestro sistema tenga conocimiento de las variables del entorno. Para evitar tener que ejecutar siempre los mismos comandos constantemente cada vez que iniciamos el contenedor o generamos una nueva terminal, debemos ejecutar lo siguiente:

1. Configurar el archivo `~/.bashrc` para que automáticamente establezca las variables de entorno en nuestro espacio de trabajo `ros2_ws`:

```
nano ~/.bashrc
```

2. Dentro de este archivo en la última línea escribiremos lo siguiente:

```
source /opt/ros/humble/setup.bash
source /workspace/ros2_ws/install/local_setup.bash
export LIBGL_ALWAYS_SOFTWARE=1
export LIBGL_DEBUG=verbose
```

3. Guardaremos el archivo actualizado. No obstante, esto no asegura que cada vez que ejecutemos de nuevo el contenedor, esta actualización se haya guardado. Esto se debe a que el contenedor se resetea cada vez que se cierra para evitar actualizarse automáticamente y que pueda que haya incompatibilidades en algún momento con algún cambio que hagamos, de esta forma, si hay algo que hemos tocado que haga una incompatibilidad, con cerrar y volver a lanzar el contenedor sería suficiente. En este caso, sí que nos interesa que el cambio se mantenga, por tanto, abriremos una nueva terminal (sin haber salido y cerrado el contenedor) y ejecutaremos el siguiente comando:

```
docker commit ros_humble ros_humble:latest
```

Y ya estaría guardado para siempre. (**NOTA:** Esto sirve a nivel genérico como se ha explicado para hacer que el contenedor mantenga todos los cambios. En el caso de archivos que se van a generar para las prácticas, esto no es necesario porque los archivos que se generen estarán guardados en nuestro dispositivo local y no íntegramente en nuestro contenedor únicamente.)

Por último, para que los cambios se guarden correctamente, es necesario cerrar el contenedor y volver a lanzarlo.

Ejemplo de un entorno

En esta primera parte nos vamos a servir de los tutoriales básicos que trae la instalación de ROS2 para entender el funcionamiento básico de `nodos` y `topics`. Ejecuta las siguientes instrucciones dentro del contenedor de `Docker` generado anteriormente:

- Terminal 1: En esta terminal lanza el nodo `/talker`. Observa el mensaje que está publicando dicho nodo. A continuación, observa la lista de nodos que existen en el entorno y estudia el entorno de ROS2 creado.

```
ros2 run demo_nodes_cpp talker
```

README

> Pregunta 1: ¿Qué podemos saber de este nodo? (Suscriptores, topics)
>
> Pregunta 2: ¿Qué podemos saber sobre el topic que publica el nodo que acabamos de lanzar?

- Terminal 2: El siguiente paso es crear un suscriptor al topic que está publicando en el nodo `/talker` (`/chatter`). Lanzaremos otro nodo, el nodo `/listener`:

```
ros2 run demo_nodes_py listener
```

> Pregunta 3: ¿Qué podemos saber sobre este nuevo nodo?
>
> Pregunta 4: ¿Qué ha sucedido ahora en el topic /chatter?

Dentro de ROS2 existen multitud de herramientas ya implementadas para ayudarnos a conocer mejor el entorno en el que se está trabajando. Entre ellas se encuentra la herramienta rqt. Dentro de `rqt` tenemos otra herramienta que nos permite ver la interconexión entre los diferentes nodos, `rqt_graph`. Para lanzarla, solo tenemos que escribir en una nueva Terminal lo siguiente:

- Terminal 3:

```
ros2 run rqt_graph rqt_graph
```

Esto nos muestra un gráfico donde aparecen los nodos y los topics que están actualmente en el entorno de ROS2.

> Pregunta 5: ¿Qué podemos saber a partir de este gráfico?

Servicios

A continuación, se va a crear un servicio que nos va a sumar dos enteros. Para ello copia la carpeta `servicio_suma` en `~/ros2_ws/src`. Seguidamente, ejecuta `colcon build` desde la terminal, dentro del directorio `~/ros2_ws`. De esta forma ya habremos creado el servicio que nos va a permitir sumar dos enteros. Ahora lanza los siguientes comandos:

NOTA: Debes hacer que los archivos `add_two_ints_server.py` y `add_two_ints_client.py` sean ejecutables usando el comando `chmod u+x`.

- Terminal 1:

```
ros2 run servicio_suma service
```

- Terminal 3:

```
ros2 run servicio_suma client 3 7
```

> Pregunta 6: ¿Qué puedes comentar acerca de este servicio?
>
> Pregunta 7: Al lanzar el servidor y a continuación ejecutar el comando: `ros2 service list` ¿Qué servicios aparecen? ¿Cómo se ha creado el servicio `minimal_service` y cuál es su cometido?
>
> Pregunta 8: ¿Cómo puedo saber cuáles son los parámetros de la "petición" y de la "respuesta" de un servicio? ¿De qué tipología son estos parámetros? (Pista: Muéstralo con el comando `ros2 interface show`).

Ejercicios

1. Publica desde la terminal un mensaje, con la cadena de caracteres que desees al topic `/chatter`, para que este sea leído por el nodo `/listener`. Escribe los comandos que has tenido que ejecutar para que esto suceda. (Pista: Usa el comando `ros2 topic pub`)
2. Cambia el servicio `servicio_suma` para que haga la siguiente operación: `a + b - c * d / e`, siendo `a, b, c, d, e` valores enteros (`int64`) pasados por línea de comandos, el resultado debe estar en formato `float64`. Describe que has tenido que hacer y los archivos que has tenido que modificar.

Parte 2: Nivel principiante

Anteriormente se ha explicado cómo se crea un espacio de trabajo ROS2. A continuación, el objetivo es aprender a generar paquetes ROS2 y crear mensajes para que otros nodos publiquen este tipo de mensajes.

Crear un Paquete

El software en ROS2 se organiza en paquetes. Un paquete puede contener nodos ROS2, una biblioteca independiente de ROS2, un conjunto de datos, archivos de configuración, una herramienta de software de terceros, o cualquier otra cosa que lógicamente constituya un módulo útil. El objetivo de estos paquetes es proporcionar esta funcionalidad útil de una manera fácil de consumir para que el software pueda ser fácilmente reutilizado. En general, los paquetes ROS2 siguen el principio de "Ricitos de oro": suficiente funcionalidad para ser útil, pero no demasiada para que el paquete no sea pesado y difícil de usar desde otro software.

Los paquetes de ROS2 requieren de una serie de ficheros específicos para poder realizar la compilación de dicho paquete. La creación de paquetes en ROS 2 utiliza `ament` como su sistema de construcción y `colcon` como su herramienta de construcción. Puedes crear un paquete usando `CMake` o `Python`, que son los soportados oficialmente, aunque existen otros tipos de construcción.

NOTA: Aunque en la mayoría de ocasiones usaremos paquetes creados con `Python`, hay ocasiones en las que se necesiten paquetes de `CMake` por temas de funcionalidad.

Los paquetes ROS 2 Python y CMake tienen cada uno su propio contenido mínimo requerido:

`CMake`:

- `CMakeLists.txt` - archivo que describe cómo compilar el código dentro del paquete.
- `include/<package_name>` - directorio que contiene los encabezados públicos del paquete.
- `package.xml` - archivo que contiene metainformación sobre el paquete.

- `src` - directorio que contiene el código fuente del paquete.

`Python`:

- `package.xml` - archivo que contiene metainformación sobre el paquete.
- `resource/<package_name>` - archivo de marcadores para el paquete.
- `setup.cfg` - necesario cuando un paquete contiene ejecutables, para que `ros2 run` pueda encontrarlos.
- `setup.py` - contiene instrucciones sobre cómo instalar el paquete.
- `<package_name>` - un directorio con el mismo nombre que su paquete, utilizado por las herramientas ROS2 para encontrar su paquete y albergar los nodos, que contiene `__init__.py`

Los comandos para crear un nuevo paquete en ROS2 son:

```
cd /workspace/ros2_ws/src
```

`CMake`:

```
ros2 pkg create --build-type ament_cmake --license Apache-2.0 <package_name> --dependencies rclcpp
```

`Python`:

```
ros2 pkg create --build-type ament_python --license Apache-2.0 <package_name> --dependencies rclpy std_msgs
```

> Pregunta 9: Habitualmente las dependencias que necesitarán nuestros paquetes para funcionar son las vistas anteriormente, sin embargo, se pueden añadir otras ¿Qué comando me indica todas las dependencias que puedo usar en mis paquetes?

Para compilar exclusivamente el paquete creado ejecuta:

```
cd ..
colcon build --packages-select <package_name>
source install/local_setup.bash
```

Crear un mensaje

Las aplicaciones ROS2 suelen comunicarse a través de interfaces de uno de estos tres tipos: topics, servicios o acciones. ROS2 utiliza un lenguaje de descripción simplificado, el lenguaje de definición de interfaces (IDL), para describir estas interfaces. Esta descripción facilita que las herramientas ROS generen automáticamente código fuente para el tipo de interfaz en varios lenguajes de destino.

Un mensaje es una estructura de datos simple, compuesta por campos tipificados. Se admiten tipos primitivos estándar (entero, coma flotante, booleano, etc.), así como matrices de tipos primitivos. Los mensajes pueden incluir estructuras y matrices anidadas de forma arbitraria (como los structs de C).

En ROS2 el flujo de trabajo pasa por emplear paquetes de `CMake` para generar las interfaces que luego serán usadas por otros paquetes, que pueden ser también de `CMake` o `Python`. A continuación, generaremos nuestro paquete `interfaz` de la siguiente manera (Este será nuestro paquete base del cuál dependerán los otros que generarás para los ejercicios, por tanto, puedes modificarlo como necesites):

```
cd /workspace/ros2_ws/src
ros2 pkg create --build-type ament_cmake --license Apache-2.0 interfaz
colcon build --packages-select interfaz
source install/local_setup.bash
```

Los archivos `.msg` deben colocarse en el directorio llamado `msg`. Crea este directorio en `ros2_ws/src/interfaz`:

```
cd interfaz
mkdir msg
```

El directorio `msg/` contendrá el archivo `MiMensaje.msg` y dentro de este se definirá la estructura de datos del mensaje.

```
cd msg/
touch MiMensaje.msg
```

```
chmod u+x MiMensaje.msg
```

Una vez creado el fichero para crear el mensaje, lo editamos para que el mensaje tenga los siguientes campos: `int32 x`, `int32 y`, `string NOMBRE=tuNombre` (En la variable `nombre` añade el tuyo), de la siguiente manera:

```
int32 x
int32 y
string NOMBRE="tuNombre"
```

Para convertir las interfaces que has definido en código específico del lenguaje (como C++ y Python) para que puedan utilizarse en esos lenguajes, añade las siguientes líneas a `CMakeLists.txt`:

```
find_package(std_msgs REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/MiMensaje.msg"
  DEPENDENCIES std_msgs
)
```

NOTA: El primer argumento (nombre de la biblioteca) en `rosidl_generate_interfaces` debe comenzar con el nombre del paquete, por ejemplo, simplemente `${PROJECT_NAME}` o `${PROJECT_NAME}_suffix`. Consulta: ros2/rosidl#441 (comment).

Tras modificar este fichero, debemos abrir el fichero `package.xml` del paquete y añadir las siguientes dependencias:

```
<depend>std_msgs</depend>
<buildtool_depend>rosidl_default_generators</buildtool_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

Una vez hechas estas modificaciones, el siguiente paso es volver a compilar el paquete con `colcon build --packages-select interfaz`. Tras compilar correctamente el paquete, debemos actualizar las dependencias del espacio de trabajo ejecutando `source install/local_setup.bash`. Si el mensaje ha sido creado correctamente, podremos saber de qué tipo es el mensaje creado a través de la terminal.

> Pregunta 10: ¿Qué comando se debe utilizar?

Seguidamente, tendremos que crear un paquete que dependa del paquete `interfaz` que nos permita generar nodos y topics para usar el mensaje creado anteriormente, de la siguiente manera:

```
ros2 pkg create --build-type ament_python --license Apache-2.0 primer_paquete --dependencies rclpy std_msgs
colcon build --packages-select primer_paquete
source install/local_setup.bash
```

Ahora ya se puede crear un nodo que publique en un topic información con el mensaje creado. Para ello, en el directorio `primer_paquete/` de nuestro paquete vamos a crear dos nodos, un publisher, llamado `nodo_envia`, y un subscriber, llamado `nodo_recibe`.

Para crear estos dos nodos, que vamos a desarrollar en Python, ejecutamos los siguientes comandos:

```
cd /workspace/ros2_ws/src/primer_paquete/src
touch nodo_envia.py
touch nodo_recibe.py
chmod u+x nodo_envia.py
chmod u+x nodo_recibe.py
```

Y escribimos el siguiente código en cada uno de estos ficheros.

- `nodo_envia.py`:

```
import rclpy
from rclpy.node import Node

from interfaz.msg import MiMensaje

class MinimalPublisher(Node):
```

```
    def __init__(self):
        super().__init__('nodo_envia')
        self.publisher_ = self.create_publisher(MiMensaje, '/mitopic', 10)
        timer_period = 0.5
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.x = 0
        self.y = 0

    def timer_callback(self):
        msg = MiMensaje()
        msg.x = self.x
        msg.y = self.y
        self.publisher_.publish(msg)
        self.get_logger().info(f"Enviando MiMensaje: Nombre={msg.NOMBRE} x={msg.x}, y={msg.y}")
        self.x += 1
        self.y += 2


def main(args=None):
    rclpy.init(args=args)

    minimal_publisher = MinimalPublisher()

    rclpy.spin(minimal_publisher)

    minimal_publisher.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

- `nodo_recibe.py` :

```
import rclpy
from rclpy.node import Node

from interfaz.msg import MiMensaje


class MinimalSubscriber(Node):

    def __init__(self):
        super().__init__('nodo_recibe')
        self.subscription = self.create_subscription(
            MiMensaje,
            'mitopic',
            self.listener_callback,
            10)
        self.subscription

    def listener_callback(self, msg):
        self.get_logger().info(f"Recibido MiMensaje: Nombre={msg.NOMBRE} x={msg.x}, y={msg.y}")


def main(args=None):
    rclpy.init(args=args)

    minimal_subscriber = MinimalSubscriber()

    rclpy.spin(minimal_subscriber)

    minimal_subscriber.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Por último, añade la siguiente línea en `package.xml` :

```
<exec_depend>interfaz</exec_depend>
```

Y añade la siguiente línea en el archivo `setup.py` dentro de los corchetes `console_scripts` del campo `entry_points` :

```
    entry_points={
        'console_scripts': [
            'server = primer_paquete.nodo_envia:main',
            'client = primer_paquete.nodo_recibe:main'
        ],
    },
```

Compila el paquete y lanza los nodos del siguiente modo:

- Terminal 1:

```
ros2 run primer_paquete server
```

- Terminal 2:

```
ros2 run primer_paquete client
```

> Pregunta 11: Muestra el gráfico de entorno ROS generado, usando rqt_graph.
>
> Pregunta 12: Intenta modificar la constante nombre, del mensaje, modificando el código del nodo `nodo_envia`. ¿Qué sucede?
>
> Pregunta 13: ¿De qué tipo es el topic que aparece?

Ejercicios

1. Crea un paquete nuevo que se llame `p2pkg`. A continuación, en el paquete `interfaz` define un mensaje llamado `P2pkg_mensaje.msg` con los siguientes campos:

```
int32 numero
geometry_msgs/Pose posicion
string fecha
```

Una vez definido, muestra los campos del mensaje.

2. En el paquete `p2pkg`, crea un nodo publisher, llamado `nodopub_ejercicio2`, y un nodo subscriber, llamado `nodosub_ejercicio2`, que sean capaces de enviar y recibir el tipo de mensajes a través un topic al que llamaremos `/topic_ejercicio2`. Estos nodos deben ser creados dentro del paquete ROS p2pkg.

El nodo `nodopub_ejercicio2` debe tener un argumento de entrada que se asignará al campo numero del mensaje, si no se le pasa ningún valor de entrada, el valor por defecto debe ser 5 (Pista: Usa el modo estándar de ROS2). Además, para enviar todos los valores del campo posición utiliza la función *random* del paquete *random* de Python.

Ambos nodos deben mostrar por la terminal el valor del campo fecha, del campo numero, del campo `posicion.position.x` y del campo `posicion.orientation.w`. En el caso del campo fecha, debes asignarle la fecha del día actual para que se muestre por pantalla.

```
# Pista:
from random import random
posicion.orientation.x = random()
```

NOTA: Ten en cuenta que para que funcione, debes añadir en el paquete `interfaz` la dependencia `geometry_msgs`.

Parte 3: Pasando a nivel Intermedio

Crear un archivo launch

El sistema de lanzamiento (`launch`) en ROS2 se encarga de ayudar al usuario a describir la configuración de su sistema y, a continuación, ejecutarla tal y como se ha descrito. La configuración del sistema incluye qué programas ejecutar, dónde ejecutarlos, qué argumentos pasarles y las convenciones específicas de ROS2 que facilitan la reutilización de componentes en todo el sistema, asignándoles a cada uno una configuración diferente. También se encarga de supervisar el estado de los procesos iniciados y de informar y/o reaccionar ante los cambios en el estado de dichos procesos.

Los archivos `launch` escritos en `XML`, `YAML` o `Python` pueden iniciar y detener diferentes nodos, así como activar y actuar sobre diversos eventos. Consulta: Uso de XML, YAML y Python para archivos de launch de ROS2 para obtener una descripción de los diferentes formatos. El paquete que proporciona este marco es `launch_ros`, que utiliza el marco de lanzamiento no específico de ROS subyacente.

Los nodos que se han lanzado anteriormente, han tenido que ser lanzados de forma independiente en terminales separadas. Sin embargo, los podemos lanzar a la vez simplemente generando un archivo `.launch`. Por tanto, es necesario generar el directorio `launch/` que albergará los archivos de esta naturaleza, de la siguiente manera:

```
cd /workspace/ros2_ws/src/primer_paquete
mkdir launch && cd launch
touch mi_launch.py
chmod u+x mi_launch.py
```

A continuación, introducimos el siguiente código en el fichero `mi_launch.py`:

```
#!/usr/bin/env python3
from launch import LaunchDescription
from launch_ros.actions import Node


def generate_launch_description():
    return LaunchDescription([
        Node(
            package='primer_paquete',
            executable='server',
            name='nodo_envia',
        ),
        Node(
            package='primer_paquete',
            executable='client',
            name='nodo_recibe',
        )
    ])
```

Existen dos formas para lanzar el archivo:

- Dentro del paquete:

```
cd /workspace/ros2_ws/src/primer_paquete/launch
ros2 launch mi_launch.py
```

- De forma directa:

```
ros2 launch <package_name> <launch_file_name>
```

Para los paquetes con archivos `launch`, es recomendable añadir una dependencia `exec_depend` del paquete `ros2launch` en el archivo `package.xml` de su paquete:

```
<exec_depend>ros2launch</exec_depend>
```

También debes modificar el archivo `setup.py` del siguiente modo y volver a compilar el paquete:

```
import os
from glob import glob

data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        (os.path.join('share', package_name, 'launch'),
            glob('launch/*.py')),
    ],
```

Esto ayuda a garantizar que el comando `ros2 launch` esté disponible después de compilar el paquete. También garantiza que se reconozcan todos los formatos de archivo de inicio.

> Pregunta 14: ¿Con que nombre se han lanzado los nodos?

Acciones

En cualquier gran sistema basado en ROS, hay casos en los que a algún usuario le gustaría enviar una solicitud a un nodo para realizar alguna tarea, y también recibir una respuesta a la solicitud. Esto se puede conseguir a través de los `servicios` ROS.

Sin embargo, si el servicio tarda mucho tiempo en ejecutarse, el usuario podría querer disponer de la posibilidad de cancelar la solicitud durante la ejecución u obtener información periódica acerca de cómo está progresando la solicitud. Las acciones en ROS permiten crear servicios que ejecuten objetivos de larga duración que puedan ser adelantados. También proporciona una `interfaz de cliente` para enviar solicitudes al `servidor`. El `cliente` y el `servidor` se comunican mediante un protocolo desarrollado para este fin, basado en mensajes ROS.

En esta parte, se va a aprender la manera en la que se programa una acción, tanto la parte del `servidor` como la parte del `cliente`. Esta acción va a calcular la serie de Fibonacci de una longitud n.

De nuevo, nos serviremos del paquete `interfaz` para crear las interfaces necesarias para desarrollar la acción. Antes de crear una acción es importante definir los mensajes de objetivo (`goal`), realimentación (`feedback`) y resultado (`result`). Estos mensajes se generan automáticamente desde un fichero `*.action`, que debe ir dentro del directorio `action` del paquete `interfaz`, con lo cual debemos crearlo. Para este ejercicio vamos a crear un fichero llamado `Fibonacci.action` definido de la siguiente manera (Recuerda hacer ejecutable este archivo):

```
int32 orden # Objetivo
---
int32[] secuencia_final # Resultado
---
int32[] secuencia_actual # Feedback
```

Para que se generen estos mensajes, debemos añadir al `CMakeLists.txt`, antes de `ament_package()`, lo siguiente:

```
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "action/Fibonacci.action"
)
```

Y en el fichero `package.xml`:

```
<buildtool_depend>rosidl_default_generators</buildtool_depend>
<depend>action_msgs</depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

Compila el paquete para que se efectuen los cambios. Ten en cuenta que debemos depender de `action_msgs`, ya que las definiciones de acciones incluyen metadatos adicionales (por ejemplo, ID de objetivos).

Por convención, los tipos de acción irán precedidos por el nombre de su paquete y la palabra `action`. Por lo tanto, cuando queramos hacer referencia a nuestra nueva acción, tendrá el nombre completo `interfaz/action/Fibonacci`.

Podemos comprobar que nuestra acción se ha creado correctamente con la herramienta de línea de comandos:

```
source install/local_setup.bash
ros2 interface show interfaz/action/Fibonacci
```

Ahora debemos crear el paquete llamado `mi_accion` como ya has visto anteriormente, donde crearemos el servidor y el cliente de la acción. Una vez creados los mensajes que se van a intercambiar servidor y cliente, veamos cómo se crea un servidor. Para ello, crea un fichero llamado `fibonacci_action_server.py` y copia el siguiente código:

```
import time

import rclpy
from rclpy.action import ActionServer
from rclpy.node import Node

from interfaz.action import Fibonacci


class FibonacciActionServer(Node):

    def __init__(self):
        super().__init__('fibonacci_action_server')
        self._action_server = ActionServer(
            self,
            Fibonacci,
            'fibonacci',
            self.execute_callback)
```

```
    def execute_callback(self, goal_handle):
        self.get_logger().info('Executing goal...')

        feedback_msg = Fibonacci.Feedback()
        feedback_msg.secuencia_actual = [0, 1]

        for i in range(1, goal_handle.request.orden):
            feedback_msg.secuencia_actual.append(
                feedback_msg.secuencia_actual[i] + feedback_msg.secuencia_actual[i-1])
            self.get_logger().info('Feedback: {0}'.format(feedback_msg.secuencia_actual))
            goal_handle.publish_feedback(feedback_msg)
            time.sleep(1)

        goal_handle.succeed()

        result = Fibonacci.Result()
        result.secuencia_final = feedback_msg.secuencia_actual
        return result


def main(args=None):
    rclpy.init(args=args)

    fibonacci_action_server = FibonacciActionServer()

    rclpy.spin(fibonacci_action_server)


if __name__ == '__main__':
    main()
```

La línea 10 define una clase `FibonacciActionServer` que es una subclase de `Node`. La clase se inicializa llamando al constructor `Node`, nombrando nuestro nodo `fibonacci_action_server`:

```
super().__init__('fibonacci_action_server')
```

En el constructor también instanciamos un nuevo servidor de acciones:

```
self._action_server = ActionServer(
    self,
    Fibonacci,
    'fibonacci',
    self.execute_callback)
```

Un servidor de acciones requiere cuatro argumentos:

1. Un nodo ROS 2 al que añadir el cliente de acciones: `self`.
2. El tipo de acción: `Fibonacci` (importado en la línea 5).
3. El nombre de la acción: `fibonacci`.
4. Una función de devolución de llamada para ejecutar los objetivos aceptados: `self.execute_callback`. Esta devolución de llamada **debe** devolver un mensaje de resultado para el tipo de acción.

Probemos a ejecutar nuestro servidor de acciones, (**NOTA**: ya sabes de antes que debes modificar el archivo `setup.py` para que esto suceda y compilar el paquete):

```
ros2 run mi_accion server
```

> Pregunta 15: ¿Cómo podemos saber que se ha lanzado la acción correctamente? ¿Qué método me permite saber que la acción ha finalizado exitosamente?
>
> Pregunta 16: Sin crear un nodo, ¿cómo lanzarías la acción Fibonnaci? ¿Cómo observarías el resultado de esta acción?

Ahora que ya hemos creado el servidor, el siguiente paso es crear el cliente para poder lanzar la acción desde un nodo. Por ello, vamos a crear un fichero llamado `fibonacci_action_client.py` donde copiaremos el siguiente código:

```
import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node
```

```
from interfaz.action import Fibonacci


class FibonacciActionClient(Node):

    def __init__(self):
        super().__init__('fibonacci_action_client')
        self._action_client = ActionClient(self, Fibonacci, 'fibonacci')

    def send_goal(self, order):
        goal_msg = Fibonacci.Goal()
        goal_msg.orden = order

        self._action_client.wait_for_server()

        self._send_goal_future = self._action_client.send_goal_async(goal_msg, feedback_callback=self.feedback_callba

        self._send_goal_future.add_done_callback(self.goal_response_callback)

    def goal_response_callback(self, future):
        goal_handle = future.result()
        if not goal_handle.accepted:
            self.get_logger().info('Goal rejected :(')
            return

        self.get_logger().info('Goal accepted :)')

        self._get_result_future = goal_handle.get_result_async()
        self._get_result_future.add_done_callback(self.get_result_callback)

    def get_result_callback(self, future):
        result = future.result().result
        self.get_logger().info('Result: {0}'.format(result.secuencia_final))
        rclpy.shutdown()

    def feedback_callback(self, feedback_msg):
        feedback = feedback_msg.feedback
        self.get_logger().info('Received feedback: {0}'.format(feedback.secuencia_actual))


def main(args=None):
    rclpy.init(args=args)

    action_client = FibonacciActionClient()

    action_client.send_goal(20)

    rclpy.spin(action_client)


if __name__ == '__main__':
    main()
```

Ahora, vamos a lanzar el nodo para comprobar que, efectivamente, este nodo lanza una acción para que el servidor nos genere una secuencia de Fibonacci de orden 20 (**NOTA**: Recuerda modificar el archivo `setup.py` y compilar el paquete):

```
ros2 run mi_accion client
```

> Pregunta 17: Si paramos el nodo a mitad de la ejecución de la acción, ¿se cancela la acción? ¿Se puede cancelar la acción desde la terminal?

Ejercicios

1. Genera un fichero llamado `launch_ejercicio3.py` que sea capaz de lanzar los nodos creados en el ejercicio anterior al mismo tiempo. El argumento de entrada se debe añadir por línea de comandos y tener un valor por defecto de 7 (Tienes que ver cómo hacer que el valor pasado por línea de comandos se guarde como argumento en el `launch`).
2. Crea otro fichero llamado `launch_ejercicio4.py` que agrupe estos dos nodos dentro de un mismo grupo, llamado `miGrupo`. Haz uso de la etiqueta `remapping` para que el topic también se encuentre dentro de ese grupo. (Pista: `namespace`, `remapping`)
3. Genera una nueva acción, `ejFibonacci.action` donde el `goal` y el `result` sean el mismo que `Fibonacci.action`, pero el `feedback` sea de tipo float64.

4. Crea un fichero `ejercicios_fibServer.py` que sea el servidor de la acción `ejFibonacci` para que se comporte igual que `fibonacci_action_server.py`, con la diferencia de que el `feedback` ha de ser la **raiz cuadrada de la media de la secuencia de Fibonacci** que se está calculando en cada iteración.

5. Crea un fichero `ejercicios_fibClient.py` que sea el nodo cliente de la acción, para que el orden de la secuencia sea modificable a través de un parámetro de entrada. Este nodo cliente publica la frase 'en proceso' a un topic llamado `/estado_accion` mientras la acción esté en proceso. Finalmente, debe mostrar el resultado obtenido de la acción por pantalla.

6. Crea un paquete llamado `service_temp`, este paquete deberá albergar un servicio de ROS 2. Este servicio se encargará de realizar conversiones de temperatura, concretamente de Grados Celcius a Farenheit y viceversa. El funcionamiento es el siguiente:
   - `Servidor`: Recibe el valor de temperatura y realiza la conversión.
   - `Cliente`: Envía el valor de temperatura y el tipo de conversión a realizar y muestra el resultado obtenido desde el servidor. Ten en cuenta que el valor de temperatura deberá especificarse desde la línea de comandos del terminal al lanzar el nodo correspondiente.

Para realizar este ejercicio, se recomienda que revises la documentación de ROS 2 acerca de los Servicios, para que puedas entender las diferencias respecto a ROS Noetic, así mismo revisa como crear tus propios archivos `.srv`.

En este caso, el archivo `.srv` debería tener el siguiente contenido:

```
# Request
float64 input_temp
string conversion_type # Cel_to_Far o Far_to_Cel
---
# Response
float64 converted_temp
```

7. Implementa un sistema basado en acciones ROS 2 utilizando Python, para ello deberás crear un nuevo paquete llamado `battery_act`, puedes encontrar la info necesaria en el siguiente enlace y en el siguiente. Esta acción se va a encargar de simular un proceso de descarga de la batería de un robot, enviando información sobre el progreso y devolviendo un resultado al finalizar. El objetivo es establecer un valor de batería en el cual el robot debe enviar un aviso de "batería baja" para proceder a su carga. El funcionamiento de la acción sería el siguiente:
   - El nodo `Servidor` llamado `battery_charger`, recibirá el `Goal` que será el valor de batería, en tanto porcentual, en el cual el robot deberá mandar el aviso (Ej: 20%). Por tanto, el robot partirá de un 1005 de batería y esta se irá reduciendo en un 5% por cada segundo que pase. El `Servidor` deberá ir actualizando el valor del `Feedback` mostrando el valor actual de la batería del robot. Asimismo, el `Servidor` devolverá como `Result` un aviso (Ej: "Batería Baja, por favor cargue el robot!"). Ten en cuenta que la acción debe poder cancelar si se manda la petición de cancelación.
   - El nodo `Cliente` llamado `battery_client`, enviará el `Goal` al servidor teniendo en cuenta que este valor se recogerá desde la línea de comandos al lanzar el nodo. El nodo `Cliente` deberá ir publicando el `Feedback` de la acción, así como el mensaje del `Result` una vez haya finalizado la acción.

En base al ejercicio, el archivo `.action` deberá tener el siguiente contenido:

```
# Goal
int32 target_percentage
---
# Result
string warning
---
```

Releases

No releases published

Packages

No packages published

Languages

- Python 59.2%
- Dockerfile 27.0%
- Shell 13.8%